

Stabilization Analyses

12/03/2024

Description

This file enables stabilization analyses for the feature value estimations. In other words, feature values are not estimated for a single, pre-defined number of characters, but are estimated for discrete steps (defined with “stepsize”), and for a given maximum number of characters (defined as “n”).

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages(“ggplot2”)`. For the Hrate package this is different, since it comes from github. The devtools library needs to be installed, and then the `install_github()` function is used.

```
library(stringr)
library(ggplot2)
library(plyr)
library(entropy)
library(ggExtra)
library(gsubfn)
```

```
## Loading required package: proto
```

```
# library(devtools)
# install_github("dimalik/Hrate")
library(Hrate)
```

Load Data

Read the preprocessed files with character strings per row.

```
# give file paths to the files to be processed
teddi <- read.csv("~/Github/UpperPalCombinatorics/data/TeDDi/final_sample.csv")
signbase <- read.csv("~/Github/UpperPalCombinatorics/data/signBase/signBase_randomized.csv")
cdli.urukV <- read.csv("~/Github/UpperPalCombinatorics/data/CDLI/UrukV/signs_clean.csv")
cdli.urukIV <- read.csv("~/Github/UpperPalCombinatorics/data/CDLI/UrukIV/signs_clean.csv")
cdli.urukIII <- read.csv("~/Github/UpperPalCombinatorics/data/CDLI/UrukIII/signs_clean.csv")
```

Preprocessing

```
# TeDDi
# select relevant columns
teddi.selected <- teddi[c("filename", "text", "writing_system")]
```

```

# rename columns
colnames(teddi.selected) <- c("id", "sequence", "subcorpus")
# add column which identifies the corpus
teddi.selected$corpus <- rep("TeDDi", nrow(teddi.selected))

# SignBase
signbase <- signbase[signbase$archaeological_unit == "Aurignacien", ]
signbase.selected <- signbase[c("object_id", "archaeological_unit", "site_name", "coding_clean")]
colnames(signbase.selected) <- c("id", "corpus", "subcorpus", "sequence")

# Proto-Cuneiform
## Uruk V
cdli.urukV.selected <- cdli.urukV[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukV.selected) <- c("id", "sequence")
cdli.urukV.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukV.selected))
cdli.urukV.selected$subcorpus <- rep("Uruk V", nrow(cdli.urukV.selected))
## Uruk IV
cdli.urukIV.selected <- cdli.urukIV[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukIV.selected) <- c("id", "sequence")
cdli.urukIV.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukIV.selected))
cdli.urukIV.selected$subcorpus <- rep("Uruk IV", nrow(cdli.urukIV.selected))
## Uruk III
cdli.urukIII.selected <- cdli.urukIII[c("tablet.id", "signs.expand.vec")]
colnames(cdli.urukIII.selected) <- c("id", "sequence")
cdli.urukIII.selected$corpus <- rep("Proto-Cuneiform", nrow(cdli.urukIII.selected))
cdli.urukIII.selected$subcorpus <- rep("Uruk III", nrow(cdli.urukIII.selected))

# Combined Data Frame
data.combined <- rbind(teddi.selected, signbase.selected, cdli.urukV.selected,
                       cdli.urukIV.selected, cdli.urukIII.selected)

```

Stabilization Analysis per Sequence

```

# set counter
counter = 0
# set the maximal number of characters (max.n), the minimum number (min.n),
# and the stepsize for stabilization analysis
# (i.e. in steps of how many units are values calculated?).
# Note: it should be the case that min.n >= stepsize
max.n = 200
min.n = 100
stepsize = 10
# initialize dataframe to append results to
stabilization.df <- data.frame(id = character(0), corpus = character(0),
                               subcorpus = character(0), sequence = character(0),
                               huni.chars = numeric(0), hrate.chars = numeric(0),
                               ttr.chars = numeric(0), rm.chars = numeric(0),
                               num.char = numeric(0))

# start time
start_time <- Sys.time()
for (i in 1:nrow(data.combined))

```

```

{
  try({ # if the processing fails for a certain file, there will be no output for this file,
    # but the try() function allows the loop to keep running

    # get meta information per row
    row.data <- data.combined[i, ]
    id <- row.data$id
    corpus <- row.data$corpus
    subcorpus <- row.data$subcorpus
    sequence <- row.data$sequence
    num.char <- nchar(row.data$sequence)

    # if/else statement to distinguish between the proto-cuneiform signs
    # and individual UTF-8 characters in the other sequences
    if (corpus == "Proto-Cuneiform"){
      chars <- unlist(strsplit(sequence, " "))
    } else {
      chars <- unlist(strsplit(sequence, ""))
    }

    # preprocess text with bag-of-words normalization
    # note: this is an important step! Without it, entropy rate estimation will fail
    # to yield realistic results due to issues with processing special characters.
    chars <- PreprocessText(chars)

    # maximal length in chars
    chars <- chars[1:max.n] # use only maximally n units
    # remove NAs for vectors which are already shorter than max.n
    chars <- chars[!is.na(chars)]

    # use "next" statement to exclude files with less than x characters
    if (length(chars) < min.n) {
      next
    }

    # run loop with stepsizes
    # define the number of units (i.e. characters) used for analyses (note that k is
    # always either equal to or smaller than n)
    k = length(chars)
    for (j in 1:(k/stepsize))
    {
      # unigram entropy estimation
      # define substring of chars vector by stepsize
      sub.chars <- chars[1:(j*stepsize)]
      # calculate unigram entropy for characters
      chars.df <- as.data.frame(table(sub.chars))
      # print(chars.df)
      huni.chars <- entropy(chars.df$Freq, method = "ML", unit = "log2")
      # entropy rate estimation
      # note: the values chosen for max.length and every.word will crucially
      # impact processing time. max.length = NULL means all units in the file are
      # considered.
      hrate.chars <- get.estimate(text = sub.chars, every.word = 1,

```

```

                                max.length = NULL)
# calculate type-token ratio (ttr)
ttr.chars <- nrow(chars.df)/sum(chars.df$Freq)

# calculate repetition measure according to Sproat (2014)
# the overall number of repetitions is the sum of frequency counts minus 1.
R <- sum(chars.df$Freq)-1
# calculate the number of adjacent repetitions
r = 0
if (length(sub.chars) > 1){
  for (m in 1:(length(sub.chars)-1)){
    if (sub.chars[m] == sub.chars[m+1]){
      r = r + 1
    } else {
      r = r + 0
    }
  }
  # calculate the repetition measure
  rm.chars <- r/R
} else {
  rm.chars <- "NA"
}

# append results to dataframe
local.df <- data.frame(id, corpus, subcorpus, sequence,
                        huni.chars, hrate.chars,
                        ttr.chars, rm.chars, num.char = j*stepsize)
stabilization.df <- rbind(stabilization.df, local.df)
}
# counter
counter <- counter + 1
print(counter)
})
}

```

```

## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
## [1] 11
## [1] 12
## [1] 13
## [1] 14
## [1] 15
## [1] 16
## [1] 17
## [1] 18
## [1] 19

```

```
## [1] 20
## [1] 21
## [1] 22
## [1] 23
## [1] 24
## [1] 25
## [1] 26
## [1] 27
## [1] 28
## [1] 29
## [1] 30
## [1] 31
## [1] 32
## [1] 33
## [1] 34
## [1] 35
## [1] 36
## [1] 37
## [1] 38
## [1] 39
## [1] 40
## [1] 41
## [1] 42
## [1] 43
## [1] 44
## [1] 45
## [1] 46
## [1] 47
## [1] 48
## [1] 49
## [1] 50
## [1] 51
## [1] 52
## [1] 53
## [1] 54
## [1] 55
## [1] 56
## [1] 57
## [1] 58
## [1] 59
## [1] 60
## [1] 61
## [1] 62
## [1] 63
## [1] 64
## [1] 65
## [1] 66
## [1] 67
## [1] 68
## [1] 69
## [1] 70
## [1] 71
## [1] 72
## [1] 73
```

```
## [1] 74
## [1] 75
## [1] 76
## [1] 77
## [1] 78
## [1] 79
## [1] 80
## [1] 81
## [1] 82
## [1] 83
## [1] 84
## [1] 85
## [1] 86
## [1] 87
## [1] 88
## [1] 89
## [1] 90
## [1] 91
## [1] 92
## [1] 93
## [1] 94
## [1] 95
## [1] 96
## [1] 97
## [1] 98
## [1] 99
## [1] 100
## [1] 101
## [1] 102
## [1] 103
## [1] 104
## [1] 105
## [1] 106
## [1] 107
## [1] 108
## [1] 109
## [1] 110
## [1] 111
## [1] 112
## [1] 113
## [1] 114
## [1] 115
## [1] 116
## [1] 117
## [1] 118
## [1] 119
## [1] 120
## [1] 121
## [1] 122
```

```
end_time <- Sys.time()
end_time - start_time
```

```
## Time difference of 14.86693 secs
```

```
head(stabilization.df)
```

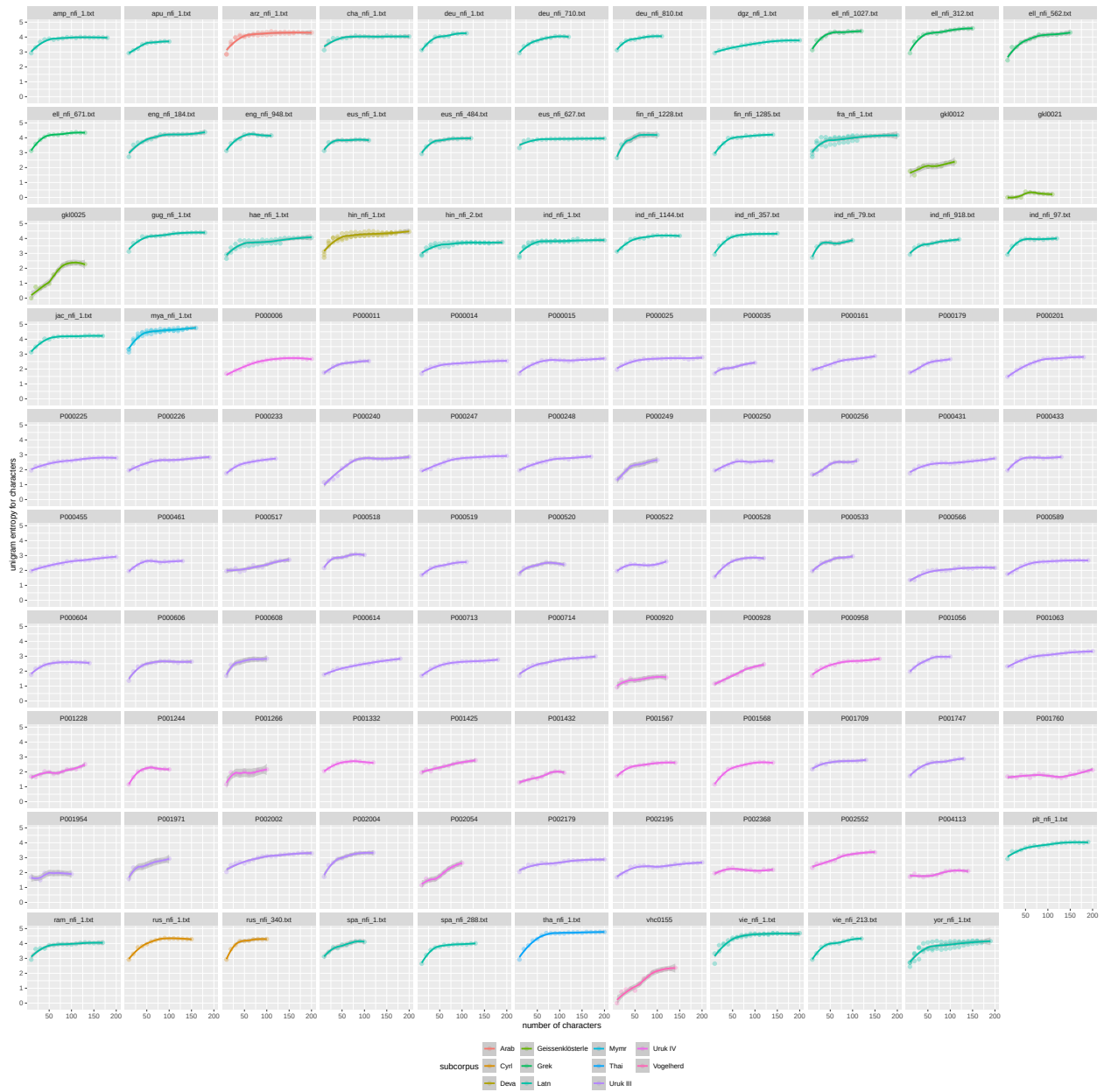
```
##           id corpus subcorpus
## 1 gug_nfi_1.txt TeDDi      Latn
## 2 gug_nfi_1.txt TeDDi      Latn
## 3 gug_nfi_1.txt TeDDi      Latn
## 4 gug_nfi_1.txt TeDDi      Latn
## 5 gug_nfi_1.txt TeDDi      Latn
## 6 gug_nfi_1.txt TeDDi      Latn
##
## 1 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
## 2 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
## 3 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
## 4 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
## 5 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
## 6 Pe táva guasu oñemboja'o mbohapy hendápe , ha táva oĩva guive ko yvy ári ho'apa ; ha Tupã imandu'a
##   huni.chars hrate.chars ttr.chars rm.chars num.char
## 1   3.121928   2.029106 0.9000000      0      10
## 2   3.684184   2.494454 0.7000000      0      20
## 3   3.840224   2.604142 0.5666667      0      30
## 4   4.131687   2.850401 0.5250000      0      40
## 5   4.103465   2.695611 0.4400000      0      50
## 6   4.161139   2.675362 0.4000000      0      60
```

Stabilization plots

Unigram entropy characters

```
huni.chars.plot <- ggplot(stabilization.df, aes(x = num.char, y = huni.chars,
                                                color = subcorpus)) +
  geom_point(alpha = 0.3, size = 2) +
  geom_smooth() +
  theme(legend.position = "bottom") +
  labs(x = "number of characters", y = "unigram entropy for characters") +
  facet_wrap(~id)
huni.chars.plot
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Safe figure to file

```
ggsave("~/Github/UpperPalCombinatorics/figures/stabilization_huni_chars.pdf", huni.chars.plot, dpi = 300,
        width = 20, height = 20,
        scale = 1, device = cairo_pdf)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

Entropy rate characters

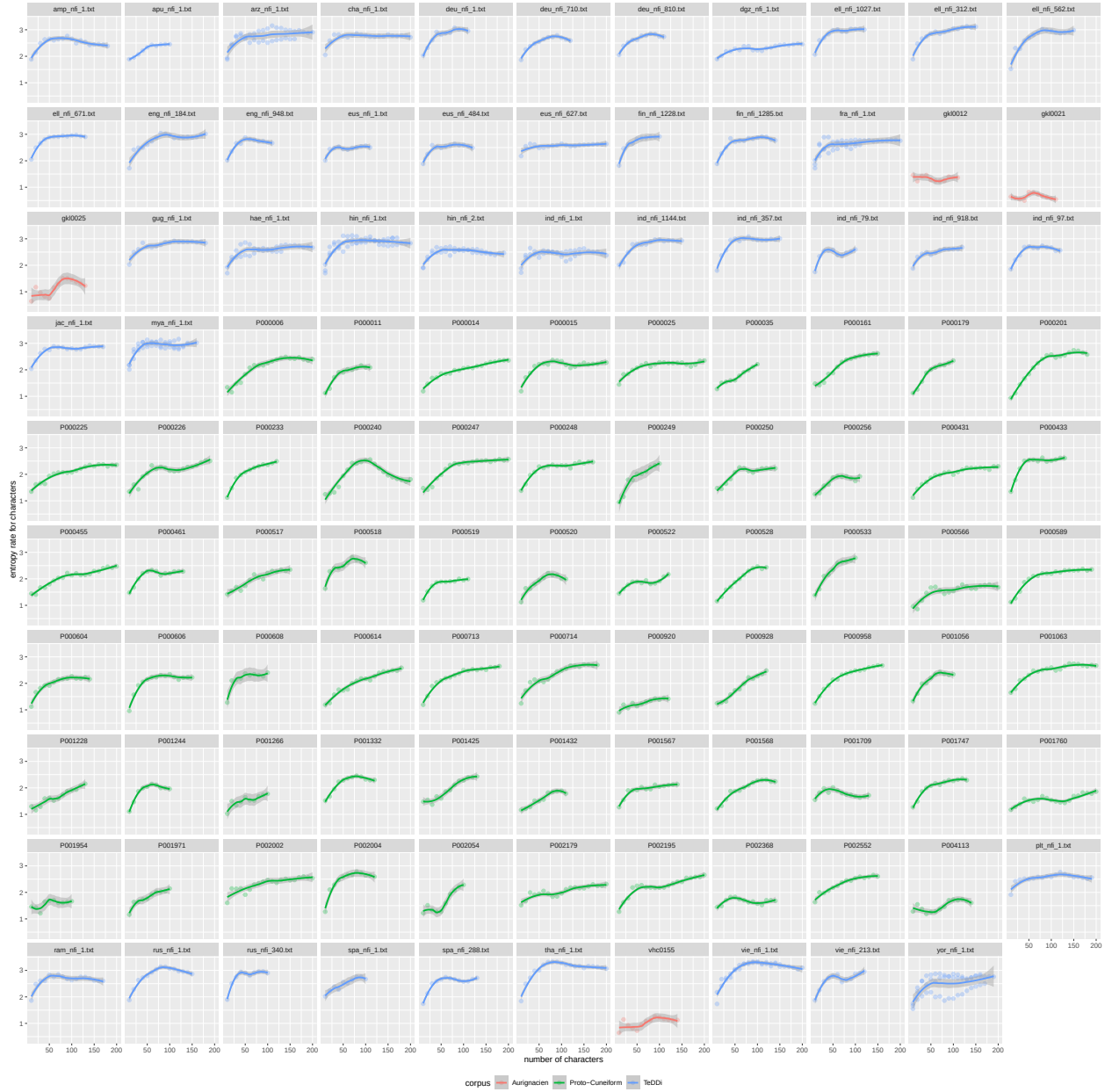

```

hrate.chars.plot <- ggplot(stabilization.df, aes(x = num.char, y = hrate.chars,
                                                colour = corpus)) +

  geom_point(alpha = 0.3, size = 2) +
  geom_smooth() +
  theme(legend.position = "bottom") +
  labs(x = "number of characters", y = "entropy rate for characters") +
  facet_wrap(~id)
hrate.chars.plot

```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Safe figure to file

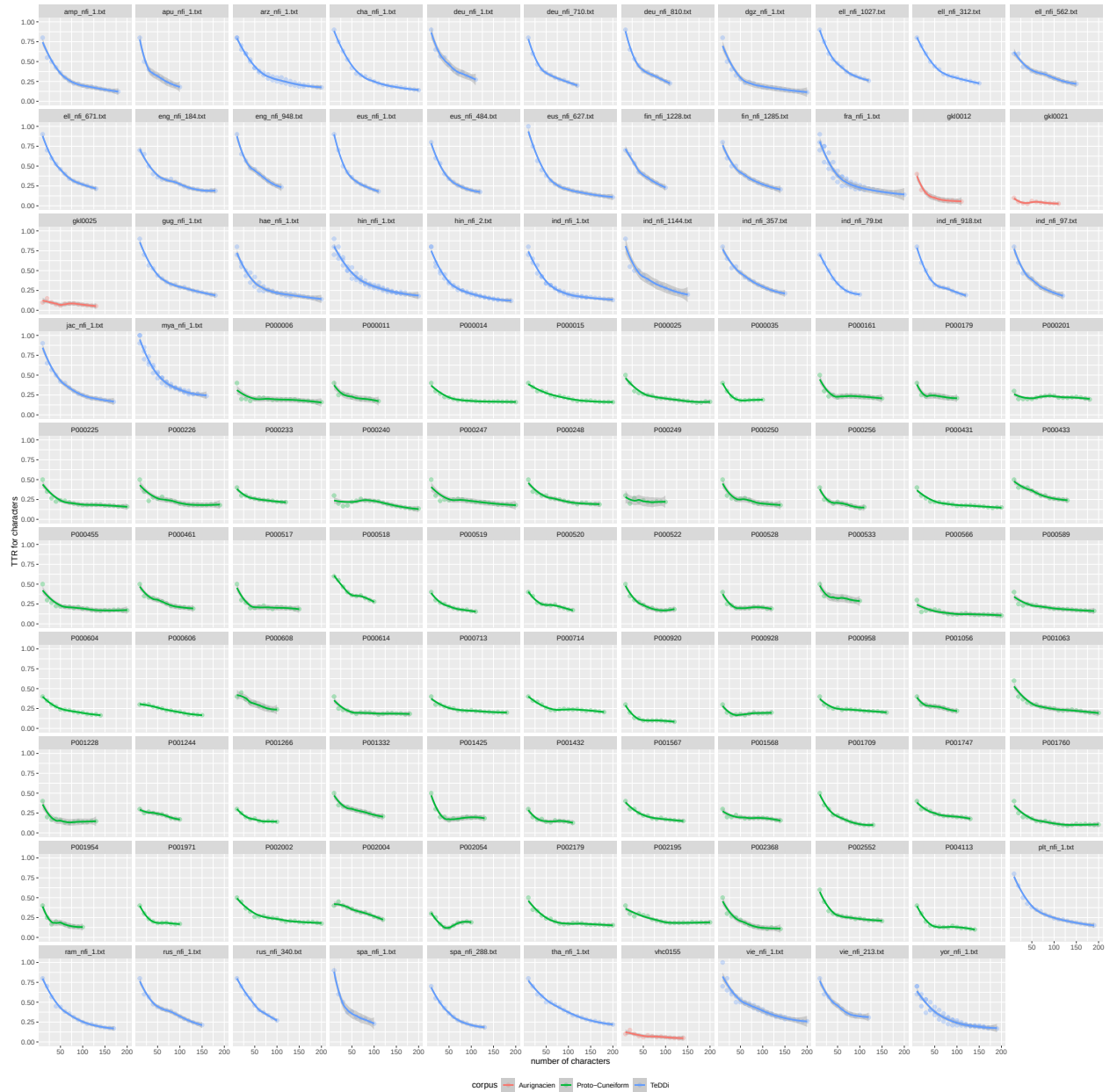
```
ggsave("~/Github/UpperPalCombinatorics/figures/stabilization_hrate_chars.pdf", hrate.chars.plot, dpi = 300,
        width = 20, height = 20,
        scale = 1, device = cairo_pdf)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

TTR characters

```
ttr.chars.plot <- ggplot(stabilization.df, aes(x = num.char, y = ttr.chars,
                                                colour = corpus)) +
  geom_point(alpha = 0.3, size = 2) +
  geom_smooth() +
  theme(legend.position = "bottom") +
  labs(x = "number of characters", y = "TTR for characters") +
  facet_wrap(~id)
ttr.chars.plot
```

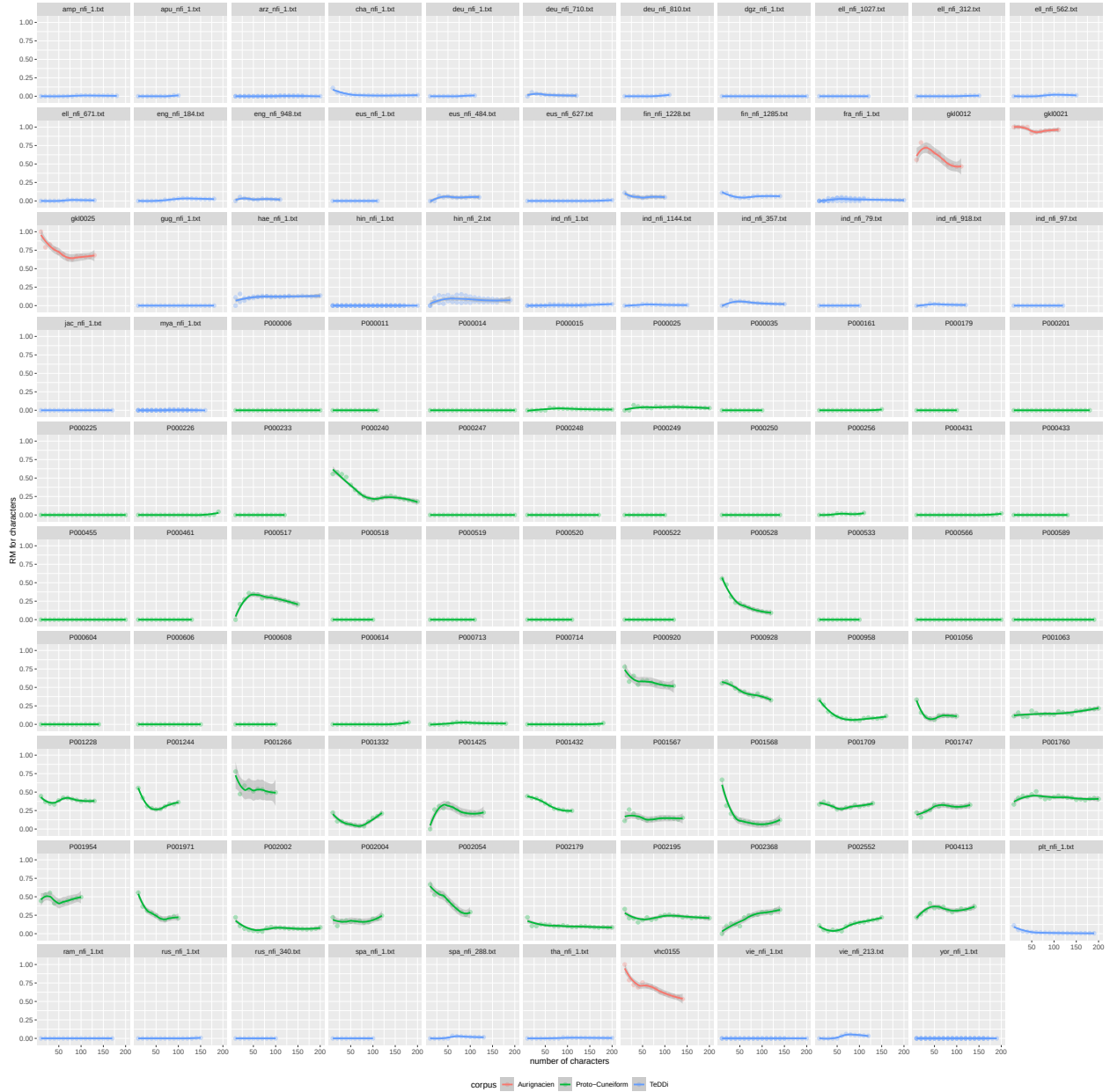
```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



```
rm.chars.plot <- ggplot(stabilization.df, aes(x = num.char, y = rm.chars,
                                              colour = corpus)) +

  geom_point(alpha = 0.3, size = 2) +
  geom_smooth() +
  theme(legend.position = "bottom") +
  labs(x = "number of characters", y = "RM for characters") +
  facet_wrap(~id)
rm.chars.plot
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Save figure to file

```
ggsave("~/Github/UpperPalCombinatorics/figures/stabilization_rm_chars.pdf", rm.chars.plot, dpi = 300,  
        width = 20, height = 20,  
        scale = 1, device = cairo_pdf)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```